Архитектура и работа пайплайна Factmap

Состояние на 2026-10-27, ветка `feat/hotel-agent-mvp`, коммит `1976cb3`. Стенд: `hotel-agent-backend:conflicts-20261027`, `hotel-agent-frontend`, PostgreSQL 16 с PostGIS 3.4. Документ описывает устройство системы и путь запроса целиком: от сообщения в чате до факта на карте.

Карта системы

Три сервиса Compose и один том данных:

Сервис	Что делает	Порт
`db`	PostgreSQL 16 с PostGIS 3.4: отели, здания, факты, геометрии доказательств, чат	5432
`migrate`	dbmate применяет миграции и завершается; бэкенд стартует только после успеха	—
`backend`	FastAPI, вся логика, агент, геометрия, отдача тайлов	8000
`frontend`	nginx с собранным React-приложением и локальными тайлами базовой карты	8080

Реестр характеристик `characteristics/` и данные `data/` в образ не попадают: они монтируются только для чтения. Все порты слушают `127.0.0.1`.

Слои бэкенда

Код живёт в `backend/src/hotel_agent/` и разложен по четырём слоям; импорты идут только внутрь.

Слой	Что в нём	Что ему запрещено
`domain`	Сущности факта, характеристики, запроса; оценка совпадения, разрешение конфликтов, затухание уверенности	Знать про HTTP, SQL, провайдеров
`application`	Сценарии (`use_cases/`), порты (`ports/`), агентный цикл (`agent/`)	Импортировать фреймворк, драйвер БД, SDK провайдера
`adapters`	PostGIS, DEM и рельеф, OSM, веб-исследование, поставщики, модель, эмбеддинги, погода	Импортировать `interfaces`
`interfaces`	HTTP-роутеры и CLI	Импортировать `adapters`

Связь между слоями — 41 порт в `application/ports/`. Конкретные реализации выбираются один раз в `main.py` и `composition.py`; контроллеры вызывают сценарии, а не хранилище. Контракт импортов проверяется `import-linter` в `make lint`.

Ключевое архитектурное решение: **у `LLMPort` нет метода, который вернул бы значение характеристики**. Модель умеет только разбирать запрос, планировать вызов инструмента, писать вопрос отельеру и раскладывать свободный ответ по полям с цитатой. Значение приходит из инструмента, из расчёта или от отельера.

Откуда берутся данные

Загрузка идёт в четыре шага, и только последние два нужны жюри.

Шаг	Команда	Что делает	Сеть
1	`make fetch-seed`, `make fetch-russia`	Качает сырой экстракт OSM и тайлы DEM в `data/raw/`	Да
2	`make build-seed`	Режет из сырых данных срез Сочи в `data/seed/sochi`	Нет
3	`make seed`	Грузит готовый срез в PostGIS: отели, здания, берег, пляжи, POI, граф пешеходных путей	Нет
4	`make precompute`	Считает факты по загруженному срезу	Нет

В репозитории лежит результат шага 2, поэтому клон начинается сразу с `make up && make seed && make precompute`. Национальный слой (`make ingest-russia`) грузится потоково из полного снапшота OSM и в репозиторий не входит.

`make precompute` считает пять групп фактов:

1. **Высота здания** — по тегу `height`, иначе по `building:levels`, иначе по типу здания.
2. **Шум по сторонам здания** — балл 0–10 по расстоянию до дорог с учётом класса, железной дороги и баров, отдельно для каждого фасада.
3. **Фасады** — контур здания разбивается на стороны с азимутом; это основа для шума, вида и этажа.
4. **Пеший путь до моря** — маршрут по графу улиц OSM до ближайшей точки доступа к берегу, не по прямой.
5. **Вид на закат** — дуга азимутов заката за апрель — август считается из положения солнца, фасады с отклонением до 45° от дуги считаются закатными, профиль горизонта по DEM и высотам соседних зданий даёт минимальный этаж. Факт сохраняется с объяснением вида «Закат в апрель — август садится по азимуту 284–303°. Горизонт открывается с 4-го этажа».

Схема хранения

Таблица	Что хранит
`hotels`, `buildings`, `facades`, `rooms`	Объекты и их геометрия
`facts`	Значение, единица, уточнения, провенанс, статус, ссылки на геометрию, объяснение
`geo_features`	Геометрия доказательства: луч видимости, маршрут, объект рядом
`conflicts`	Набор противоречащих фактов, правило, победитель или состояние «нерешено»
`hotelier_questions`	Вопрос агента, причина, статус, ответ и дата ответа
`chat_*`	Переписка, заголовок и снимок выдачи для восстановления экрана

Провенанс лежит колонками, а не в JSON: `source_type`, `source_id`, `source_url`, `method`, `confidence`, `computed_at`, `observed_at`, `valid_until`, `evidence`. Частичный уникальный индекс допускает ровно один

активный факт на пару «объект + характеристика», поэтому новое значение вытесняет прежнее через статус, а не затиранием.

Пайплайн запроса

Один запрос проходит восемь стадий. Каждая сообщает о себе событием `progress` в поток SSE, и интерфейс показывает их по мере готовности.

1. Чат решает, это разговор или поиск

`Chat.send` отдаёт сообщение диалоговому слою. Консультация и вопрос про выбранный отель получают ответ текстом без запуска пайплайна. Поиск продолжает путь. Прежний контекст переписки и снимок выдачи сохраняются, поэтому уточнение не теряет условия предыдущего запроса.

2. Разбор запроса

Модель превращает фразу в структуру: пожелания, объекты рядом с радиусом, пространственные требования, даты, число гостей, бюджет, подсказку места, исправления опечаток, признак «куда угодно» и признак «нужно уточнение». Выдуманные характеристики на офлайн-пути отбрасываются. Если места нет и «куда угодно» не стоит, пайплайн останавливается и возвращает вопрос.

3. Подбор кандидатов

Место разрешается по очереди: прямое совпадение с Сочи, затем административная область, затем bbox города из базы, затем именованный географический объект, затем геокодер. Для объекта берётся его настоящая геометрия с буфером 10 км, для точки — окно 50 км. Если места нет вовсе, кандидаты набираются семантическим поиском по документам. Регион, который стенд не обслуживает, честно возвращает пустую выдачу с пометкой.

4. Нативные инструменты

Часть пожеланий закрывается без агента: объекты рядом по тегу и радиусу, расстояния, вид из окна на названный объект, факты из предрасчёта. Это быстрый путь, он работает и без модели.

5. Агентный цикл по каждому пожеланию

Пожелания исследуются параллельно, по три одновременно, с общим сроком 300 секунд и личным сроком 150 секунд. Каждое пожелание получает собственный `WishLoop`: до 32 шагов, журнал вызовов, бюджет на каждый инструмент.

Инструменты агента:

Группа	Действия
Данные	`inspect_data`, `list_candidates`, `read_records`, `match_records`, `search_documents`
География	`find_hotels`, `find_nearby`, `measure_geometry`, `measure_terrain`, `assess_view`, `calculate_sun`
Внешние сведения	`search_web`, `read_web_page`, `get_provider_content`, `get_room_offers`
Среда и события	`get_weather`, `search_events`, `read_event_page`, `observe_wildlife`, `measure_landcover`
Служебные	`check_requested`, `finish`

Цикл устроен так, чтобы модель не могла обьявить совпадение без доказательства:

- любое предложенное совпадение проходит проверку критериев, а текстовое — требование цитаты из источника;
- `finish` без оснований возвращается обратно с замечанием, повторный такой `finish` прекращает проверку как незавершённую;
- одинаковый вызов с теми же аргументами ловится как дубль, три дубля прекращают цикл;
- шестнадцать шагов без нового результата считаются застоем, и пожелание закрывается статусом «источники ничего не дали» — это не равно «свойства нет»;
- исчерпанный бюджет инструмента и приближение срока приходят агенту как наблюдения, а не как молчаливый обрыв.

Итог пожелания — один из статусов: подтверждено, отклонено, неизвестно, нужен вопрос пользователю, недоступно.

6. Оценка и ранжирование

`score` считает долю выполненных условий: вклад каждого условия равен `вес × качество × уверенность`, где качество — полное (1.0), частичное (0.5) или ноль, а уверенность затухает к половине исходной по мере приближения срока годности. Неизвестное условие оставляет вес в знаменателе и даёт ноль в числителе: отель опускается, но не исчезает. Обязательное условие без факта убирает отель из выдачи. Факт с оговоркой — «с 4-го этажа», «для западной стороны» — принудительно понижается до частичного. Сортировка идёт по оценке, затем по числу настоящих расчётов вида, затем по имени.

7. Проверка рекомендаций

Отобранные отели проходят смысловую проверку: отклонённые уходят, а причина отклонения возвращается в исследование — пожелания, которые проверка поставила под сомнение, исследуются повторно и только они. Ответ ассистента тоже проверяется: если он расходится с выдачей, вместо него показывается безопасная формулировка.

8. Сохранение и ответ

Факты и геометрия доказательств сохраняются, ответ собирается текстом, списком отелей, совпадениями с объяснениями и ссылками на геометрию. Фронтенд рисует список слева, карточку с фактами и карту с доказательством: луч видимости, маршрут до моря, объект рядом.

Работа с моделью

Модель подключена по OpenAI-совместимому протоколу; есть адаптер Anthropic и детерминированный `FakeLLM`, на котором работают все тесты и офлайн-демо. Вокруг вызова собрано четыре механизма:

- **структурированный ответ** — схема на каждый тип запроса, а не разбор свободного текста;
- **кеш ответов** — одинаковый запрос в пределах процесса не оплачивается дважды;
- **бюджет попыток** — срок на попытку считается по наблюдаемому времени ответа модели, медленный ответ хеджируется вторым запросом, а после трёх зависаний порядок моделей меняется;
- **разделение моделей** — быстрая модель планирует шаги, основная принимает решения; запасная модель никогда не отвечает первой на вопрос-суждение.

Отельер

Агент ставит вопрос, когда данных нет или он в них не уверен; вопрос хранится с причиной. Отельер отвечает свободным текстом, модель раскладывает ответ по полям с цитатой, значение становится фактом с источником «отельер». Если такой ответ расходится с расчётом, срабатывает разрешение конфликта: правило `hotelier > fresh_computed > osm_tag > stale_computed`, запись в `conflicts`, а при близких уверенностях — состояние «нерешено» вместо тихой победы одной стороны.

Карта и геометрия

Рельеф берётся из локального DEM Copernicus GLO-30 в границах среза и из внешних тайлов за его пределами, с дисковым кешем. Базовая карта отдаётся из локальных pmtiles, слои и 3D-здания — из PostGIS. Линия видимости считается от фасада на высоте этажа через весь путь до цели с учётом рельефа, промежуточных зданий, высоты цели и кривизны Земли; неизвестная высота не превращается в «вид открыт». Сцена отеля строится из настоящих контуров вокруг него.

Фронтенд

React 19, Vite, TypeScript, маршруты лениво, слои `app → pages → features → entities → shared`. Типы API генерируются из OpenAPI, запросы идут через TanStack Query, моков нет: экран показывает только то, что вернул бэкенд, а отсутствующий метод превращается в честное состояние ошибки. Ответ чата приходит потоком SSE, и прогресс поиска виден во время ожидания. Два языка и две темы, все строки из словарей.

Что проверяется автоматически

Где	Что
Бэкенд	`ruff`, `mypy` на 555 файлах, 984 юнит-теста, 37 контрактных тестов против `openapi.yaml`, контракт импортов
Фронтенд	36 проверок правил, 560 тестов, типы, линтер, сборка
Стенд	Health-проверки Compose, миграции до старта бэкенда

Где система уязвима

Коротко; подробно и с числами — в `work/task-coverage/review-2026-10-27.md`.

- Оценка дискретна, поэтому отели часто получают одинаковый балл, и порядок внутри группы задан именем.
- Вне локального DEM рельеф, геометрия и веб-исследование зависят от сторонних сервисов без фолбэка.
- Срок годности фактов заполняется, но просроченные значения никто не перечитывает.
- Ссылка на источник у фактов из веба в базе не сохраняется.
- Объяснения фактов приходят по-русски независимо от выбранного языка.